

Checkpoint 3 - Preguntas

1. ¿Cuáles son los tipos de Datos en Python?

Los tipos de datos en Python son:

- String
- Number
- Boolean
- Bytes / bytes array
- Lists
- Tuples
- Dictionaries
- Sets
- None

2. ¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?

Para las variables en Python se recomienda usar la convención de nomenclatura establecida en la guía de estilo **PEP 8**.

Ejemplos:

- palabras descriptivas
`edad = 25`
- snake-case que son palabras en minúscula separadas por guión bajo
`nombre_usuario = 'Juan'`
- no se recomienda usar mayúsculas, como por ejemplo
`NumeroAlumnos = 25`, ya que se puede confundir con una clase en Python
- se recomienda evitar las variables de una sola letra, estas incluyen: "L" minúscula, "O" mayúscula o "I" mayúscula.
- no se pueden usar las palabras reservadas de Python, como 'class', 'lambda', 'def'...

3. ¿Qué es un Heredoc en Python?

Los Heredocs son cadenas de varias líneas y se usan las triples comillas para definirlos, como en el siguiente ejemplo

```
mensaje = """ Buenas tardes,  
Este es un mensaje de varias líneas en Python.  
Un saludo """
```

4. ¿Qué es una interpolación de cadenas?

Es una técnica que permite insertar variables dentro de un String de una manera dinámica y legible por medio de varios métodos, como por ejemplo **.format()** y **f-string** (que es el que más se usa).

Ejemplo de f-string:

```
nombre = 'Alex'  
ciudad = 'Vitoria'  
mensaje = f'Mi nombre es {nombre} y vivo en {ciudad} .
```

5. ¿Cuándo deberíamos usar comentarios en Python?

Una buena práctica es evitar la redundancia de comentarios. Deberíamos usarlos solo cuando un código no es fácil de entender a simple vista o para marcar el #TODO y así saber que tenemos que volver a él.

6. ¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?

Las **aplicaciones monolíticas** funcionan como una sola unidad, son más rápidas de desarrollar y suelen tener mejor rendimiento, pero pueden ser difíciles de mantener si crecen demasiado.

Las **aplicaciones de microservicios** están divididas en servicios independientes, lo que facilita su escalabilidad y mantenimiento, pero pueden tener problemas de comunicación y ser más complejas de gestionar.